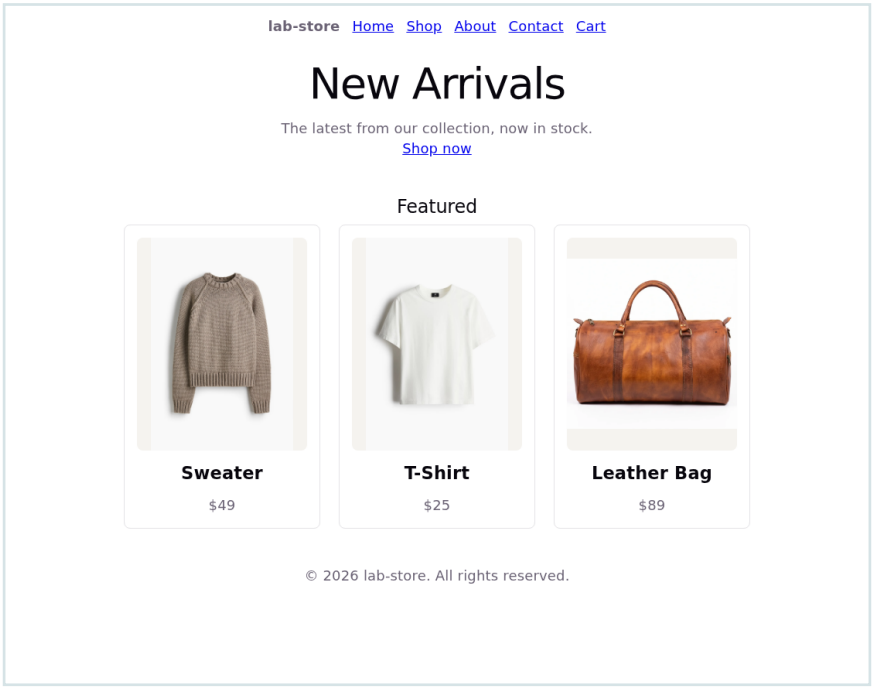
Home: tres productos, tres enlaces.

Las tarjetas ahora tienen foto y abren el detalle del producto elegido. El catálogo sigue siendo local: no hay una llamada a una API.



Home actual. Sweater, T-Shirt y Leather Bag usan el mismo componente ProductCard.

Cómo se reparten los componentes

Home muestra HeroBanner y FeaturedProducts. **FeaturedProducts** recorre el catálogo de **src/data/products.ts**. Por cada producto crea una **ProductCard**, que recibe sus datos por props.

FeaturedProducts.tsx · mismo código, con los atributos en una línea

```
{products.map((product) => (
    <ProductCard key={product.id} product={product} />
))}
```

ProductCard.tsx · fragmento; se omiten clases y aria-label para enfocar la navegación

```
<Link to={`/products/${product.id}`}>
    <img src={product.images[0].src} alt={product.images[0].alt} />
    <h3>{product.name}</h3>
    <p>${product.price}</p>
</Link>
```

Aquí no hace falta useEffect. map() prepara las tarjetas y Link resuelve la navegación. Recibir datos por props y mostrarlos en JSX no requiere un efecto.

Cómo llega el ID al detalle.

react-router-dom es una librería que se añade a React para gestionar las rutas y navegar entre pantallas. Ya está instalada en este proyecto.

useParams viene de esa librería

useParams es un hook: una función que react-router-dom ya trae para leer los parámetros de la URL actual. No lo creamos nosotros ni viene del paquete React.

1. La tarjeta coloca el ID en el enlace

ProductCard.tsx · se omite el contenido de la tarjeta

```
<Link to={`/products/${product.id}`}>
    ...
</Link>
```

Para Sweater, **product.id** vale **"sweater"**. Al hacer clic, el navegador va a **/products/sweater**.

2. El router define dónde va ese ID

App.tsx · fragmento de las rutas; se omiten las demás pantallas

```
import { Routes, Route } from 'react-router-dom';

<Routes>
    <Route
        path="/products/:productId"
        element={<ProductDetails />}
    />
</Routes>
```

Los dos puntos en **:productId** indican una parte variable de la ruta. React Router toma el valor **"sweater"** de la URL, lo llama **productId** y muestra ProductDetails.

3. El detalle lee el ID con useParams

ProductDetails.tsx · lectura del parámetro dentro del componente

```
import { useParams } from 'react-router-dom';

const { productId } = useParams();
const product = products.find(product => product.id === productId);
```

En **/products/sweater**, useParams() devuelve **{ productId: "sweater" }**. Luego find() usa ese ID para buscar el producto en nuestro catálogo.

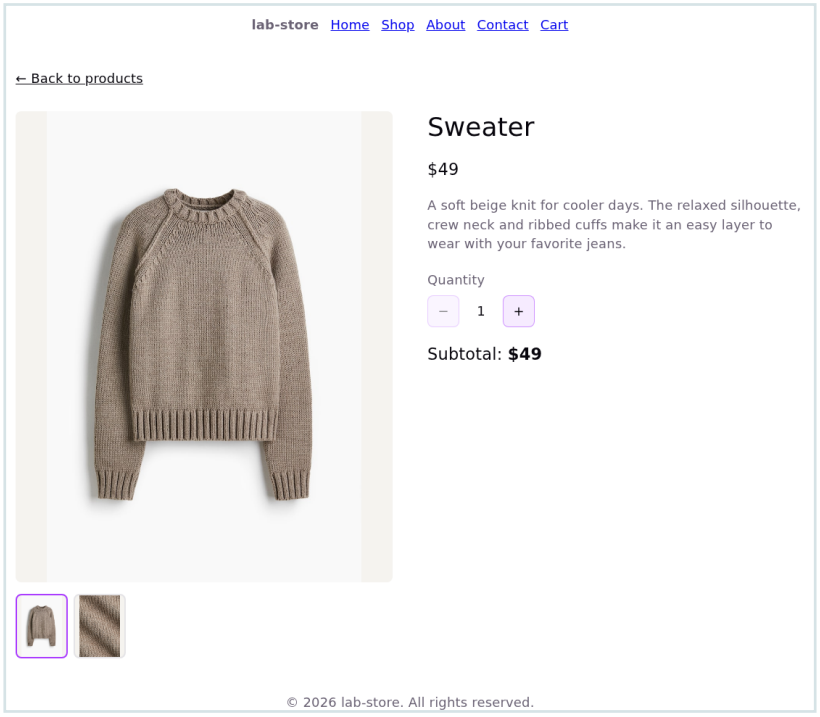
El ID viaja en la URL. Link escribe el valor, Route le da un nombre y useParams lo lee.

Documentación oficial de React Router: rutas y parámetros dinámicos · useParams.

En el proyecto, BrowserRouter ya envuelve App en main.tsx para habilitar esta navegación.

Detalle: una página por producto.

La foto y sus miniaturas quedan a la izquierda. A la derecha se muestran descripción, precio, cantidad y subtotal. En móvil, el contenido se apila.



/products/sweater. La misma estructura se usa para T-Shirt y Leather Bag.

La página busca; el componente muestra

App.tsx conecta **/products/:productId** con **ProductDetails**. La página lee el ID y busca el producto. **ProductPreview** es el componente que contiene la galería, los controles y el efecto del título.

ProductDetails.tsx · cuerpo del componente, tal como está implementado

```
const { productId } = useParams();
const product = products.find(product => product.id === productId);

if (!product) {
    return <NotFound />;
}

return <ProductPreview key={product.id} product={product} />;
```

Props: product entrega los datos. **Estado local:** ProductPreview recuerda la foto y la cantidad. **key:** identifica el producto; uno diferente recibe una nueva instancia con estado inicial.

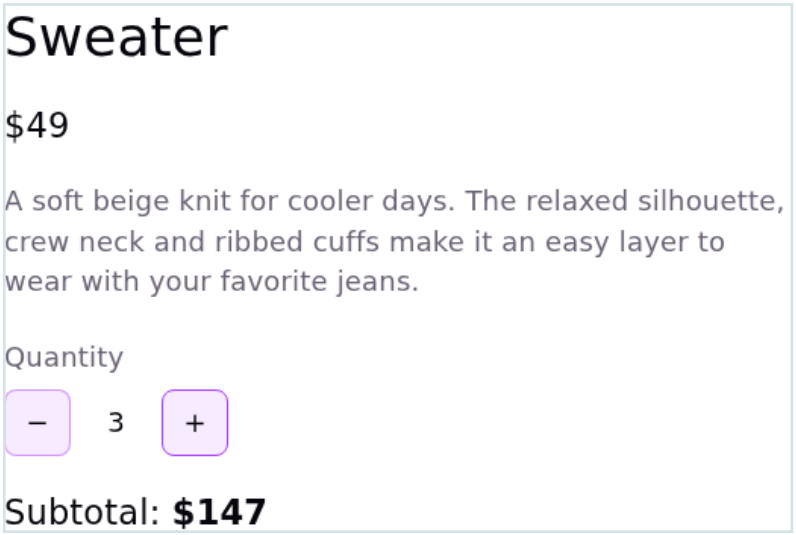
Al volver a Home se desmonta el detalle. Al abrirlo otra vez, empieza con cantidad 1 y la primera foto. La ruta Shop queda vacía; los productos se abren desde Home.

useState: recordar una elección.

La foto elegida y la cantidad deben persistir entre renders. useState guarda esos valores; su función setter solicita que React vuelva a renderizar con el nuevo estado.



Foto 2 seleccionada.



Cantidad 3: subtotal $147.

ProductPreview.tsx · valores iniciales y datos calculados

```
const [quantity, setQuantity] = useState(1);
const [selectedImageIndex, setSelectedImageIndex] = useState(0);
const selectedImage = product.images[selectedImageIndex];
const subtotal = product.price * quantity;
```

quantity es el valor actual; **setQuantity** lo actualiza. Lo mismo ocurre con **selectedImageIndex** y **setSelectedImageIndex**. El índice 0 representa la primera foto.

ProductPreview.tsx · handlers de la miniatura y del botón +, respectivamente

```
onClick={() => setSelectedImageIndex(index)}
onClick={() => setQuantity(previous => previous + 1)}
```

El clic en una miniatura cambia solo la foto. El botón + usa la cantidad anterior para sumar 1. React vuelve a ejecutar el componente y recalcula **subtotal**: 49 × 3 = 147.

El subtotal no necesita useEffect ni otro estado: se obtiene del precio y la cantidad. En React no existe **setEffect**; los setters de este ejemplo son setQuantity y setSelectedImageIndex.

useEffect: conectar con el navegador.

Existe para sincronizar un componente con algo externo a su renderizado. Por ejemplo: el título de la pestaña, un listener del navegador o una conexión.

Por qué lo usamos en este detalle

JSX describe lo que se ve dentro de la página. **document.title** controla la pestaña del navegador. El efecto lo mantiene alineado con el producto una vez que React aplica el render; así evitamos modificarlo durante el cálculo de la UI.

ProductPreview.tsx · efecto real; se omite únicamente el comentario

```
useEffect(() => {
    const previousTitle = document.title;
    document.title = `${product.name} | lab-store`;

    return () => {
        document.title = previousTitle;
    };
}, [product.name]);
```

Leelo en este orden

1. Guardar: previousTitle recuerda el título que tenía la pestaña antes del detalle.
2. Sincronizar: document.title pasa a “Sweater | lab-store”.
3. Limpiar: la función que se devuelve restaura previousTitle al salir.
4. Depender: [product.name] indica qué dato usa el efecto. Si cambia en la misma instancia, React limpia la ejecución anterior y vuelve a sincronizar.

Comprobado en la aplicación:
Home: **lab-store**
Detalle: **Sweater | lab-store**
Volver a Home: **lab-store**
Estos son los títulos leídos del navegador; no forman parte de la captura de la página.

Qué dispara el efecto y qué no

Se ejecuta al montar el detalle. Cambiar cantidad o foto vuelve a renderizar, pero **no repite este efecto**: product.name sigue igual. Al desmontar, corre la limpieza. Si cambiamos de producto con otra key, se limpia la instancia anterior y se monta una nueva.

StrictMode: en desarrollo hay un ciclo extra de efecto, limpieza y efecto. Es una comprobación de React; dejalo activado. La limpieza hace que repetir ese ciclo sea seguro.

Probalo: abrí Sweater, mirá el título de la pestaña, cambiá cantidad y foto, y volvé a Home. La UI cambia con el estado; el título se sincroniza y se restaura con el efecto.

Documentación: useEffect · useState y memoria del componente · Cuándo no hace falta un efecto